Assignment 4

An AgriTech startup wants to predict whether a farmer should grow Rice, Wheat, or Maize based on soil and climate conditions using crop recommendation dataset.

1. Understand features: N, P, K, Temperature, Humidity, Rainfall
2. Define input–output mapping
3. Identify ML problem type
4. Perform exploratory data analysis (EDA)

In [59]:
```
import pandas as pd
```

Loading Dataset

In [60]:
```
df=pd.read_csv("C:\\Users\\Lenovo\\Downloads\\Crop_recommendation.csv")
df
```

Out[60]:

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice
...	...	...	...	...	...	...	...	...
2195	107	34	32	26.774637	66.413269	6.780064	177.774507	coffee
2196	99	15	27	27.417112	56.636362	6.086922	127.924610	coffee
2197	118	33	30	24.131797	67.225123	6.362608	173.322839	coffee
2198	117	32	34	26.272418	52.127394	6.758793	127.175293	coffee
2199	104	18	30	23.603016	60.396475	6.779833	140.937041	coffee

2200 rows × 8 columns

Data Preprocessing

In [61]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   N            2200 non-null   int64  
 1   P            2200 non-null   int64  
 2   K            2200 non-null   int64  
 3   temperature  2200 non-null   float64
 4   humidity     2200 non-null   float64
 5   ph           2200 non-null   float64
 6   rainfall     2200 non-null   float64
 7   label        2200 non-null   object 
dtypes: float64(4), int64(3), object(1)
memory usage: 137.6+ KB
```

In [62]:
```
df.head()
```

Out[62]:

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

In [63]:
```
df.tail()
```

Out[63]:

	N	P	K	temperature	humidity	ph	rainfall	label
2195	107	34	32	26.774637	66.413269	6.780064	177.774507	coffee
2196	99	15	27	27.417112	56.636362	6.086922	127.924610	coffee
2197	118	33	30	24.131797	67.225123	6.362608	173.322839	coffee
2198	117	32	34	26.272418	52.127394	6.758793	127.175293	coffee
2199	104	18	30	23.603016	60.396475	6.779833	140.937041	coffee

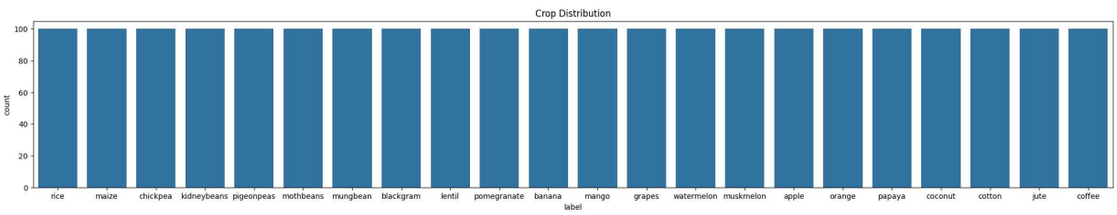
In [64]:
```
df.isnull().sum()
```

Out[64]:
```
N              0
P              0
K              0
temperature    0
humidity       0
ph             0
rainfall       0
label          0
dtype: int64
```

Graphical Representation

In [65]:
```
import matplotlib.pyplot as plt
import seaborn as sns

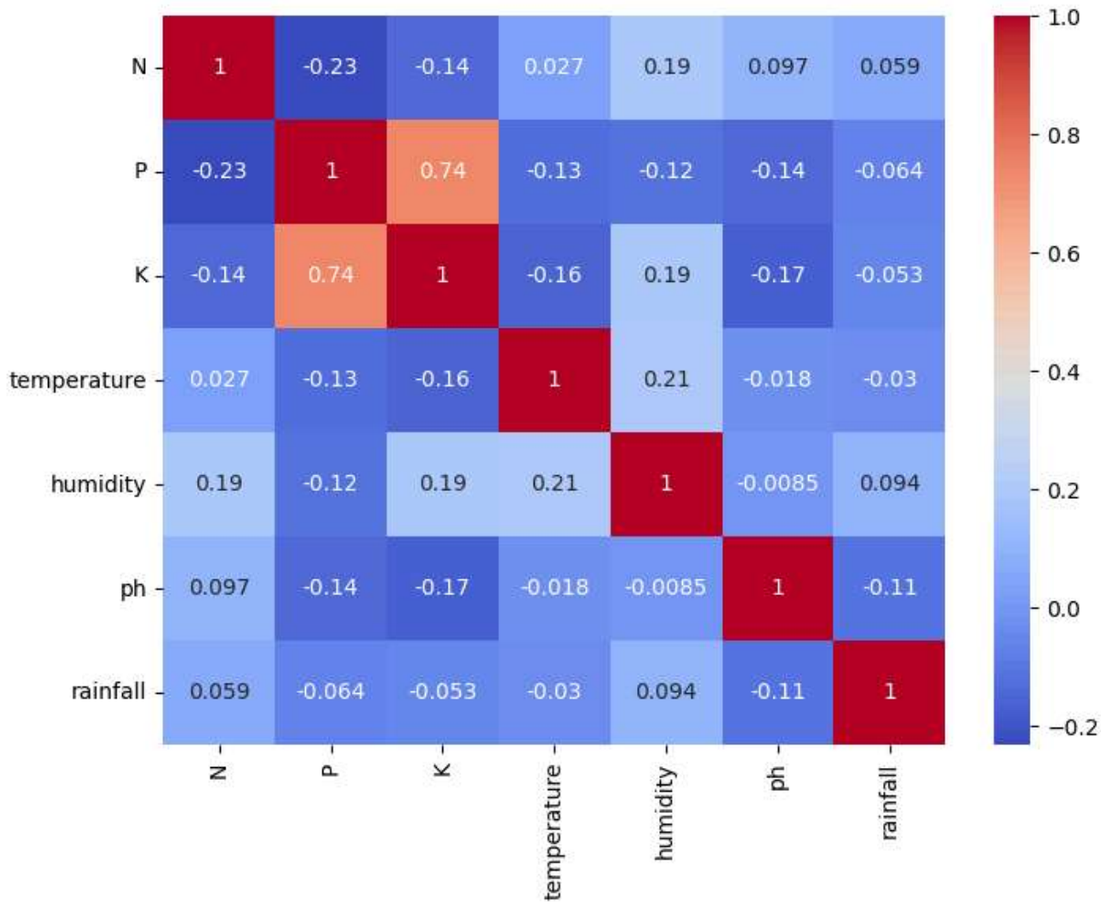
plt.figure(figsize = (26,4))
sns.countplot(x='label', data=df)
plt.title("Crop Distribution")
plt.show()
```



In [66]:
```
df1=df.drop('label',axis=1)
```

Correlation Meatrics

In [67]:
```
plt.figure(figsize=(8,6))
sns.heatmap(df1.corr(), annot=True, cmap='coolwarm')
plt.show()
```



Building Model Manually

In [68]:
```
X = df[['N','P','K','temperature','humidity','ph','rainfall']].values.tolist()
y = df['label'].values.tolist()
```

Euclidean's Distance

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

In [75]:
```
import math
from collections import Counter
import numpy as np

class ManualKNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X_train, y_train):
        self.X_train = X_train.values if hasattr(X_train, "values") else X_train
        self.y_train = y_train.values if hasattr(y_train, "values") else y_train

    def _euclidean_distance(self, p1, p2):
        distance = 0
        for i in range(len(p1)):
            distance += (p1[i] - p2[i]) ** 2
        return math.sqrt(distance)

    def predict(self, X_test):
        X_test = np.array(X_test)
        predictions = [self._predict_single(x) for x in X_test]
        return predictions

    def _predict_single(self, x):
        distances = []

        for i in range(len(self.X_train)):
            dist = self._euclidean_distance(x, self.X_train[i])
            distances.append((dist, self.y_train[i]))

        distances.sort(key=lambda x: x[0])
        k_neighbors = distances[:self.k]
```

```
        k_labels = [label for (_, label) in k_neighbors]
        return Counter(k_labels).most_common(1)[0][0]
```

In [77]:
```
model = ManualKNN(k=3)
model.fit(x, y)

model.predict([[107,34,32,26,66,177]])
```

Out[77]:
```
['coffee']
```

Conclusion

The KNN algorithm was implemented manually to classify suitable crops based on soil and climate features. Euclidean distance was used to find the nearest neighbors, and majority voting determined the final prediction. The model successfully handled the multiclass classification problem of recommending Rice, Wheat, or Maize. This approach provided a clear understanding of distance-based learning and practical application of KNN in agriculture.